

LSDE COURSEWORK

DATA SCIENCE

Name: SHIVANI MUTHUKUMARAN

Student Number: 2688016

PART 1: ArtAI Application

Design Topic: AWS Architecture for a public webservice called “ArtAI”

INTRODUCTION

ArtAI is a public web application that allows users to upload images and receive an AI generated variation of their photo, such as an enhanced or stylized version. The design of this architecture focuses on ensuring the service is globally accessible, reliable, secure and cost-efficient. I have used core AWS Services such as Amazon S3, AWS Lambda, CloudFront, API Gateway, AWS IAM, Amazon SNS and CloudWatch for the implementation.

AWS SERVICES USED AND THEIR ROLES

1. Amazon S3 (Simple Storage Service)

Two S3 buckets are used-one for hosting static website and another for storing uploaded and processed images.

- **Static hosting:** The frontend HTML, CSS, and JavaScript files of the ArtAI web application are stored in the first S3 bucket configured for static website hosting.
- **Data storage and backup:** The second bucket stores uploaded and AI-generated images. Versioning is enabled to ensure every file change creates a new version for easy file recovery. And lifecycle rules automatically move older versions to cheaper storage classes, and cross-region replication can be configured for disaster recovery.

2. Amazon CloudFront (Content Delivery Network)

To provide low-latency and globally available access, CloudFront distributes cached copies of the static website around the world. This ensures fast load time experience for users regardless of their region. It also integrates with AWS Certificate Manager (ACM) for secure HTTPS access DDoS protection through AWS Shield.

3. Amazon API Gateway

The API Gateway acts as a secure entry point for all the backend requests. When a user uploads an image, it receives the HTTP request and forwards it to the appropriate AWS Lambda function. It also supports request throttling and usage monitoring.

4. AWS Lambda

Lambda provides the backend processing of the application. When triggered, it retrieves uploaded image from S3, performs AI transformation and saves the result back to S3. It is cost-efficient because billing only occurs during function execution.

5. AWS Identity and Access Management (IAM)

IAM ensures all data and resources remain secure. Each service is assigned a dedicated IAM role. S3 bucket policies make sure that only the backend Lambda function can access the image data and public is strictly limited to CloudFront for static website content.

6. Amazon CloudWatch

CloudWatch monitors the performance metrics such as API requests and Lambda invocations. CloudWatch alarm is configured to track daily request counts. If it exceeds 10,000 in a single day, the alarm triggers an SNS notification.

7. Amazon SNS (Simple Notification Service)

SNS is used to send email notifications to the system admin when the CloudWatch alarm condition is met. This allows proactive monitoring and scaling decisions if the service experiences unusually high demand.

APPLICATION WORKFLOW

- The user accesses the ArtAI website hosted on S3 through CloudFront.
- The user uploads an image which is sent via API Gateway to AWS Lambda.
- The Lambda function retrieves the image from S3, applies AI-style effect and saves the new image to a versioned S3 bucket.
- S3 automatically keeps old versions and, if configured, replicates them to another region.
- CloudWatch monitors API requests, if more than 10000 requests occur in a single day, an SNS email is triggered to alert administrators.

ARCHITECTURE DIAGRAM

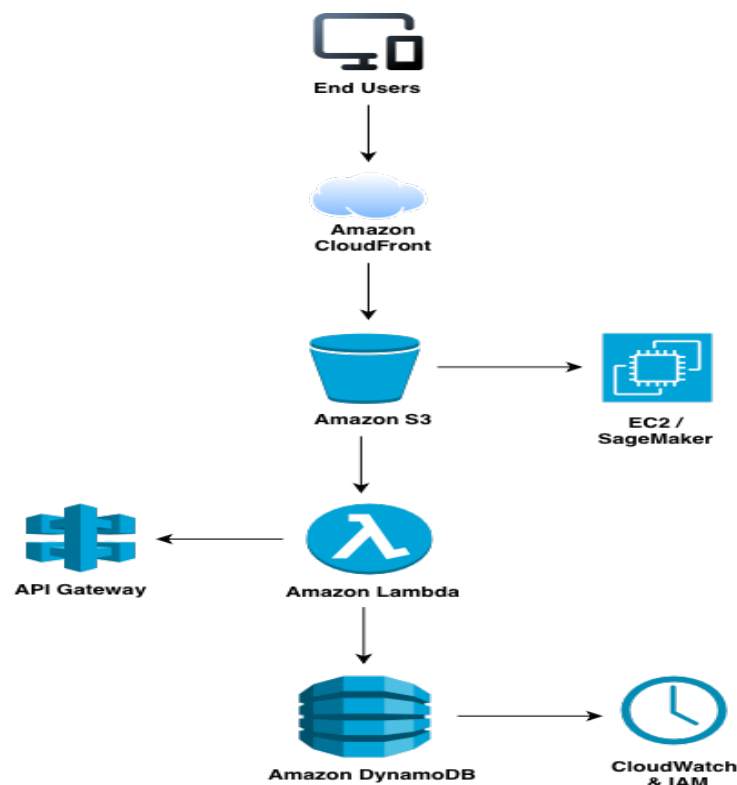


Figure 1. Architecture diagram of AWS Services used for ArtAI

CONCLUSION

This AWS Architecture for ArtAI successfully meets all requirements using foundational AWS services. It provides a scalable, secure and cost-efficient framework that supports image uploads, automated AI-style processing, versioned backups and proactive monitoring. This design demonstrates how key AWS Services can be combined to build a globally available, reliable and production ready cloud application.

PART 2: WordFreq Application

INTRODUCTION

The basic functionality of the application is to count words in a text file. It returns the top ten most frequent words found in a text document and can process multiple text files sequentially. It is a distributed word-frequency analysis workflow utilizing multiple AWS Services including EC2, S3, SQS and DynamoDB.

TASK A – Installation of the application

Infrastructure Setup

- **EC2 Development Environment:** An Ubuntu EC2 instance(t2.micro) was provisioned and configured with SSH access, a 30GB GP2 storage volume, and the EMR_EC2_DefaultRole IAM instance profile. Essential software like AWS CLI, unzip utilities, and Go 1.20.1 was installed to support application execution.
- **S3 Buckets:** Two S3 buckets were created. Uploading bucket for initial file ingestion and Processing bucket for files to be processed by the worker. Their ARNs were recorded for later workflow integration.
- **SQS Queues:** Two SQS Standard queues were configured for file-processing tasks and output messages.
- **Event Notifications:** An S3 event rule was added to the processing bucket, generating an SQS message on each object creation event, effectively linking S3 and SQS.

Application Deployment and Configuration

- **Application Package Setup:** The WordFreq application was uploaded to the uploading bucket, copied to the EC2 instance and extracted. The setup script was executed to install dependencies and create the DynamoDB Table.
- **Worker Configuration:** The worker component was configured and initial tests were conducted by purging the jobs queue, running the worker manually and uploading sample files to verify correct interactions between the services. Logs were monitored to confirm that the service processed files automatically when new objects appeared in the processing bucket.

Application Operation

The workflow begins when text files are placed into the designated uploading bucket and subsequently copied into the processing bucket. When a file arrives in the processing bucket, AWS automatically generates a notification and sends it to a jobs queue. This mechanism informs the system that a new file is ready for analysis.

Worker instances continuously monitor this jobs queue. When a notification appears, a worker retrieves the corresponding file from the processing bucket and performs the word-frequency analysis. After the file has been processed, the system records the results in a DynamoDB table, and a confirmation message is placed in the results queue.

Batch Processing Execution

Around 120 text files were uploaded and a recursive S3 copy operation transferred these files to the processing bucket, triggering the automated workflow. Continuous log monitoring on the EC2 instance verified that all files were processed successfully.

All components of the WordFreq system were successfully deployed, configured and validated. AWS services were used to build a fully operational environment capable of automatically processing text files. This setup provides necessary foundation for conducting further tasks such as auto-scaling and load-testing.

TASK B – Design and Implementation of Auto-Scaling

Architecture

The existing application was extended by introducing an EC2 Auto Scaling Group (ASG) responsible for running identical worker instances. A new Amazon Machine Image (AMI) was created to ensure that every newly launched instance had the exact application dependencies, IAM permissions and runtime environment required for immediate operation.

Because workers retrieve from SQS, no load balancer was needed. Instead, Amazon CloudWatch monitors the SQS queue to determine when additional processing capacity is required.

Scaling Policies

An EC2 Auto Scaling Group was created with a minimum and desired capacity of 1 and a maximum capacity of 9. Two CloudWatch alarms were created:

1. Scale-Out Alarm
 - Triggered when $\text{ApproximateNumberOfMessagesVisible} \geq 10$ or when $\text{ApproximateAgeOfOldestMessage} \geq 60$ seconds.
 - A step scaling policy adds one new instance, with an enforced two-minute cooldown to prevent rapid launches.
2. Scale-In Alarm
 - Triggered when $\text{ApproximateNumberOfMessagesVisible} \leq 2$ for a sustained period (e.g., 5 minutes).
 - The most recently launched instance is terminated, scaling the ASG back toward the minimum size.

This configuration ensures stable, predictable scaling behaviour and avoids flapping during periods of fluctuating load.

TASK C – Performance Testing of Auto-Scaling

Test Procedure

1. The original single worker instance created during Task A was stopped, ensuring only Auto Scaling Group instances would process jobs.
2. A batch of 120 text files was uploaded to the uploading S3 bucket.
3. A recursive copy command was executed from an EC2 instance to transfer all text files into the processing bucket.
4. The following dashboards were monitored:
 - S3 Console – file transfer progress
 - SQS Console – queue depth and in-flight messages
 - ASG Activity – instance launches and terminations
 - EC2 Instances – worker status and logs
 - DynamoDB Table – records inserted upon job completion

Observations

- As the queue began to grow, the scale-out alarm triggered and an additional worker instance was launched.
- After approximately two minutes, the new instance started consuming messages, causing queue length and message age to reduce.
- Once the queue drained and stayed below the lower threshold, the scale-in alarm triggered and the extra instance was terminated automatically.
- All the text files were processed successfully and DynamoDB showed consistent entries corresponding to completed jobs.

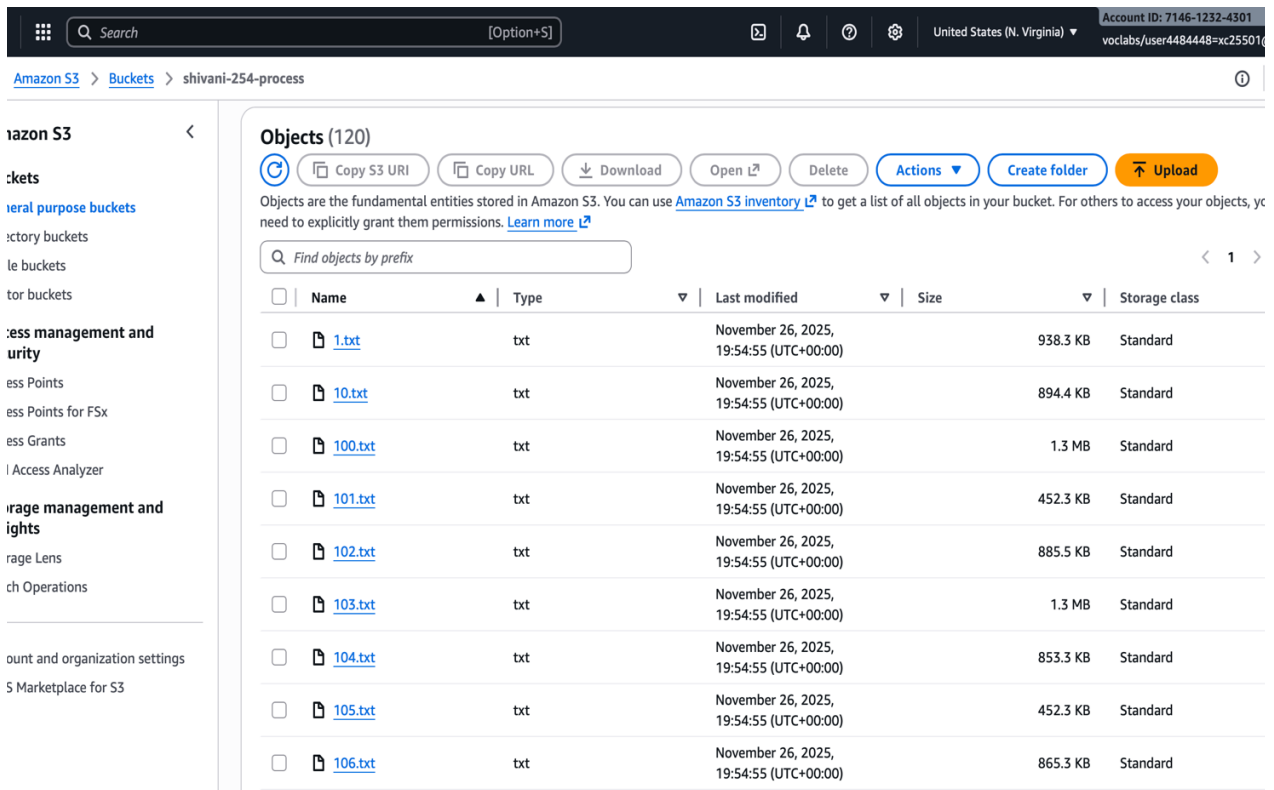


Figure 2. Copied files in the S3 processing bucket

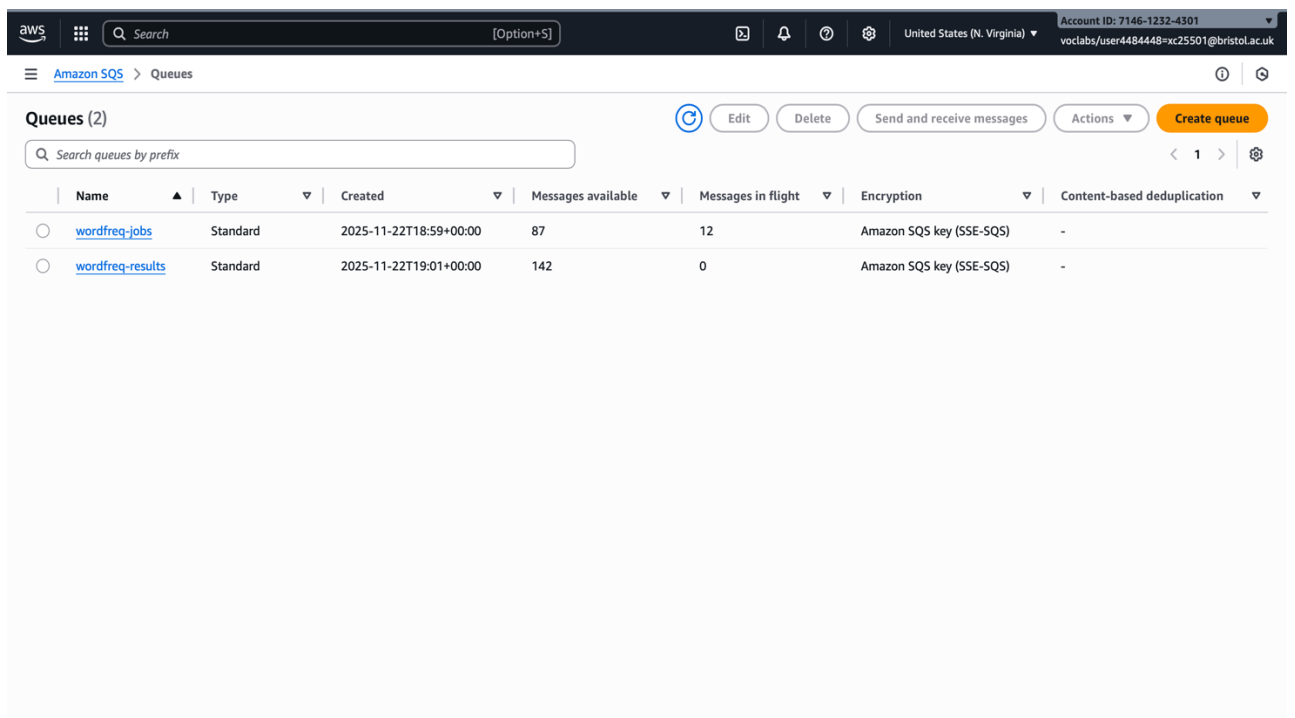


Figure 3. SQS Queues created with the message status

Amazon SQS > Queues > wordfreq-jobs > Send and receive messages

Receive messages [Info](#) [Edit poll settings](#) [Stop polling](#) [Poll for messages](#)

Messages available	Polling duration	Maximum message count	Polling progress
47	30	10	 5 receives/second

Messages (10) [View details](#) [Delete](#)

☐	ID	Sent	Size	Receive count
☐	409ee7a1-b2d8-4486-971c-2db71ea2c899	2025-11-26T19:54+00:00	783 bytes	1
☐	f96121b2-c8b0-4608-8777-836dd93802aa	2025-11-26T19:54+00:00	784 bytes	1
☐	f38729e7-7b6d-4270-8bb1-b3e15ac62772	2025-11-26T19:54+00:00	783 bytes	1
☐	d32c4349-7535-448a-b0ee-ba9cf350d2ad	2025-11-26T19:54+00:00	771 bytes	1
☐	4f5f347f-c59e-4f3c-a353-5afe14578573	2025-11-26T19:54+00:00	784 bytes	1
☐	f82574a3-1f6e-4dd2-8436-1e628306c06b	2025-11-26T19:54+00:00	771 bytes	1
☐	6de5c800-9aee-45b5-bc23-ed532c0a2cf2	2025-11-26T19:54+00:00	784 bytes	1
☐	3402c9f4-898c-4e47-a34c-25884b47ef2c	2025-11-26T19:54+00:00	770 bytes	1
☐	cdb9d00f-2af0-4947-8f8e-cd7806a42a57	2025-11-26T19:54+00:00	771 bytes	1
☐	3ab5bae4-e63f-4a74-9d49-c6758d20b141	2025-11-26T19:54+00:00	771 bytes	1

Figure 4. Messages sent and received in SQS Queue

CloudWatch > Alarms

Successfully updated alarm wordfreq-scale-in-alarm. [View alarm](#)

Alarms (2) ☐ Hide Auto Scaling alarms [Clear selection](#) [Create composite alarm](#) [Actions](#) [Create alarm](#)

Alarm state: Any Alarm type: Any Actions status: Any

☐	Name	State	Last state update (UTC)	Conditions	Actions
☐	wordfreq-scale-out-alarm	 OK	2025-11-26 18:45:38	ApproximateNumberOfMessagesVisible >= 10 for 1 datapoints within 1 minute	 Actions enabled
☐	wordfreq-scale-in-alarm	 In alarm	2025-11-26 18:53:33	ApproximateNumberOfMessagesVisible <= 2 for 1 datapoints within 5 minutes	 Actions enabled

Figure 5. CloudWatch Alarms

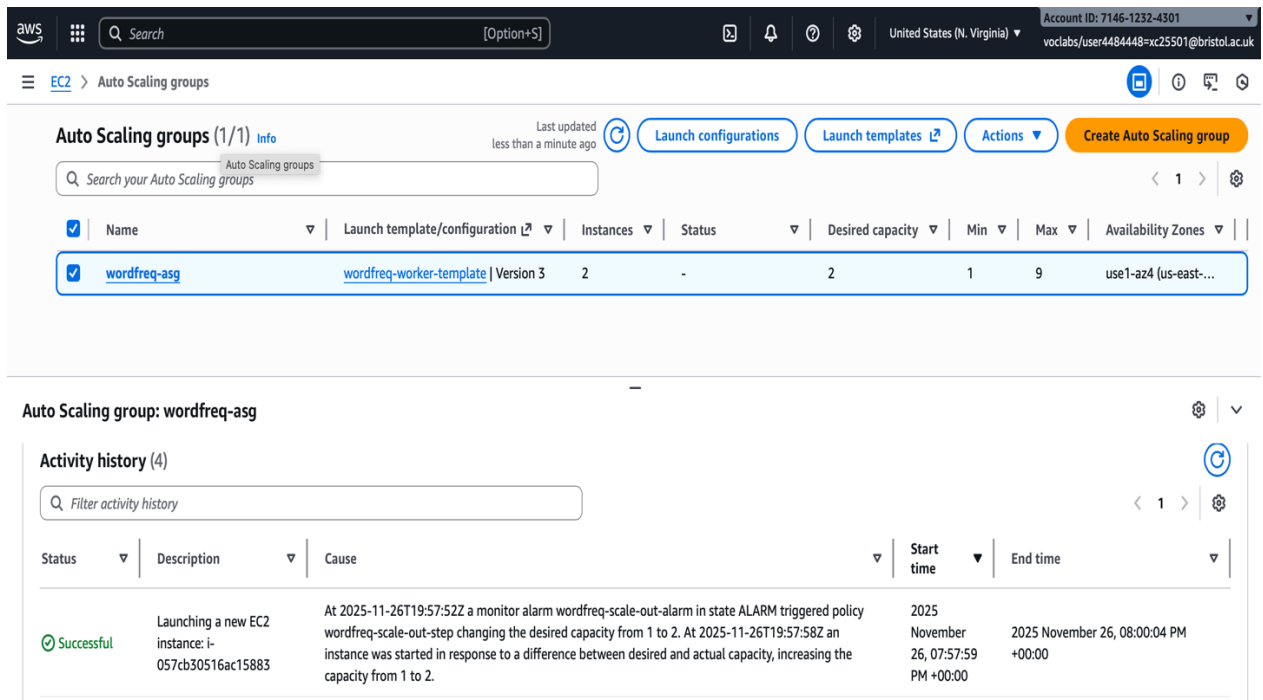


Figure 8. Instance status in Auto-scaling groups

TASK D –Optimizing the WordFreq Application

The existing cloud architecture could be improved and re-designed using AWS services without modifying the application code. The following recommendations increase resilience, security, cost-efficiency and data durability.

1. Improve Resilience and Availability

- Deploy the EC2 ASG across multiple Availability Zones to ensure the workers remain available if one AZ fails.
- Enable EC2 Auto Recovery and use CloudWatch health checks so failed instances are automatically replaced.
- Add an SQS Dead-Letter Queue (DLQ) to isolate unprocessable messages and prevent queue blockage.

2. Long-Term data Backups

- Enable DynamoDB Point-in-Time Recovery (PITR) for continuous backups.
- Turn on S3 Versioning to protect input/output files and apply Lifecycle Policies to archive older versions into S3 Glacier for low-cost long-term storage.

3. Cost-Efficient Processing for Occasional Workloads

- Use EC2 Spot Instances for workers, as the workload is non-urgent and tolerant to interruptions.
- Alternatively, schedule processing windows using EventBridge, allowing files to accumulate in SQS/S3 and be processed in periodic batches instead of immediately.
- Ensure ASG scale-in thresholds are aggressive, so the worker fleet shuts down when idle.

4. Strengthen Security and Prevent Unauthorised Access

- Apply least-privilege IAM roles, restrict worker access to only required S3 buckets, SQS queues and DynamoDB tables.
- Enable S3 Block Public Access, enforce server-side encryption and restrict bucket policies to authorized principals.
- Place EC2 workers in private subnets and restrict security groups to only essential outbound traffic.
- Enable AWS CloudTrail for auditing all API actions in the environment.

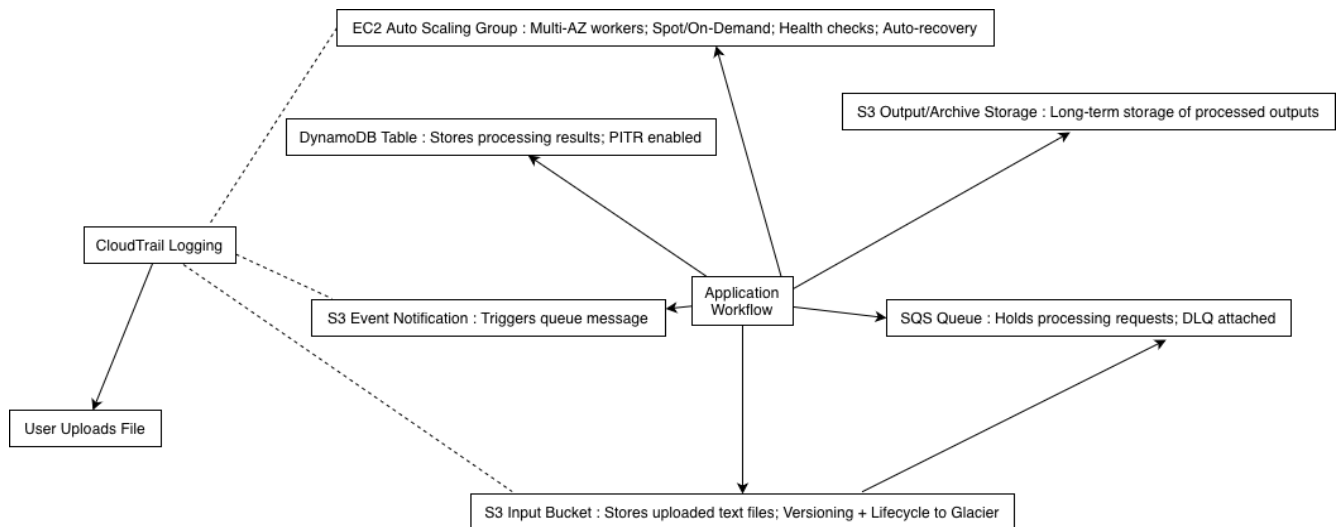


Figure 2: Architecture diagram of optimized WordFreq application

TASK E – Further Improvements

Two advanced AWS Services would significantly improve performance and robustness of the WordFreq application.

1. AWS Lambda (Serverless Processing)

AWS Lambda can process files automatically when they are uploaded to S3, removing the need for EC2 workers entirely.

Advantages:

- Fully serverless
- Automatic scaling to thousands of parallel executions.
- Pay-per-use, making it ideal for occasional workloads.
- Built-in high availability across multiple AZs.

This architecture would be much faster, cheaper and simpler than maintaining EC2 instances and Auto Scaling.

2. AWS Batch with Step Functions

For larger or more computationally heavy text processing, AWS Batch orchestrated by Step Functions offers a robust alternative.

Advantages:

- Batch automatically manages containerized compute fleets.
- Step Functions provide reliable workflow orchestration, retries and error handling.
- Capable of handling very large datasets with high parallelism.
- Highly cost-optimized and suitable for complex or long-running tasks.

This combination provides a scalable, fault-tolerant and enterprise-grade data processing pipeline.

These would act as alternative data processing services that would be far more performant and robust for this processing task.

CONCLUSION:

In summary, the technical implementation of the application provides a clear, efficient and scalable framework for processing textual data. The setup ensures consistent handling of inputs, reliable extraction of word frequencies and a structure that can be easily maintained or expanded. This foundation supports accurate analysis and positions the application for future development and optimization.

AWS ACADEMY ACCOUNT CREDENTIALS

USERNAME: xc25501@bristol.ac.uk

PASSWORD: hukgof-1jaMve-gakrip